

Worksheet 2 — Secure SDLC & Tooling (3 hrs)

Course: Software Security (KOSEN69) · **Week 2 Aligned to:** OWASP 2025 (A05 Injection [CWE-89, CWE-78], A04 Cryptographic Failures [CWE-327], A02 Security Misconfiguration [CWE-798, CWE-489]) · CWE-798, CWE-89, CWE-78, CWE-327, CWE-489 **Signature game:** "Bug Triage Race" (scan → triage; score = true positives – misclassified)

Ethics note: The scanners run only against the provided [vulnerable-repo/](#) on your own machine. Do not point SAST/secret scanners at third-party repos or production systems without authorization. Treat any secret you find here as fake lab data.

Part 1 — Student Information

Name	Student ID	Date	Group
------	------------	------	-------

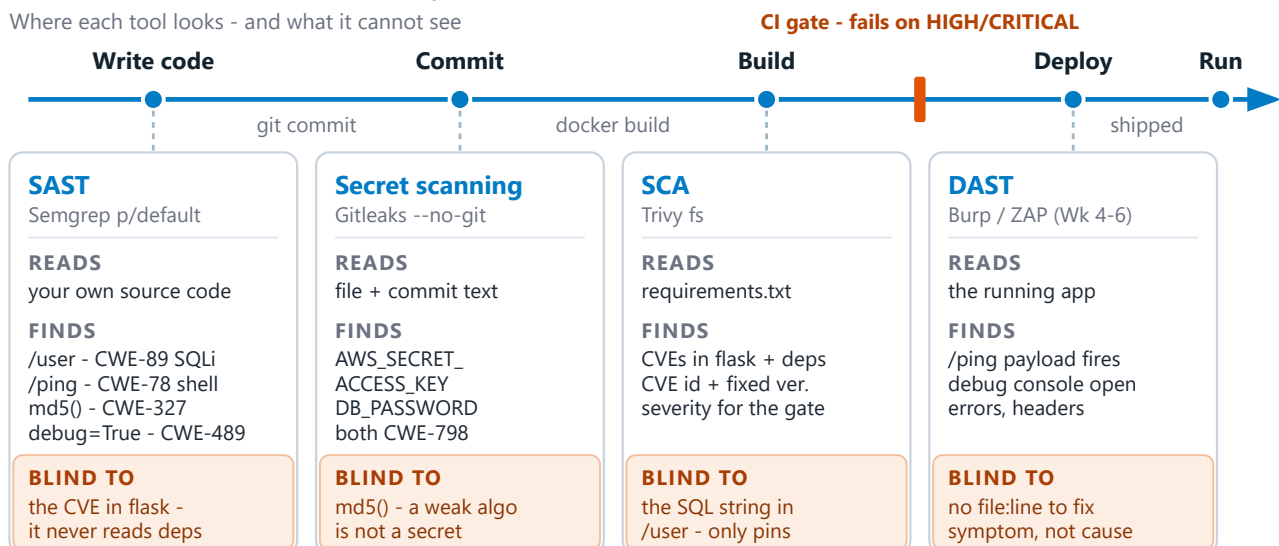
Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Distinguish SAST, DAST, and SCA — what does each see, and when in the SDLC does each run?
2. What is secret scanning, and why do hardcoded secrets keep ending up in repos?
3. What does "shift-left / DevSecOps" mean in practice for a CI pipeline?
4. Why is coverage-guided fuzzing considered the dominant modern bug-finding technique?
5. Define true positive vs. false positive in scanner triage, and why misclassifying both directions is costly.

One scanner is not "the security check"

Where each tool looks - and what it cannot see



Complements, not substitutes.

Every BLIND TO above is another card's FINDS. Task 7's logic bug is nobody's finding but yours.

+ Fuzzing (Task 4) - also at Build

Runs the compiled harness, not the text: ASan heap-buffer-overflow, harness.c:23 - no pattern for SAST to match.

Part 3 — Hands-on Lab (180 min)

Learning goals: run a SAST tool and a secret scanner, triage findings by CWE/severity, and remediate real flaws. **Prerequisites:** Docker installed; internet to pull the Semgrep/Gitleaks images.

Environment setup

```
cd labs/week02-sdlc-tooling
cat scan.sh                # see exactly what it runs
bash scan.sh               # Semgrep (p/default + p/owasp-top-ten) then Gitleaks
on ./vulnerable-repo
```

Target under scan: **vulnerable-repo/app.py** (plus **requirements.txt**). It contains five planted flaws.

What to submit per task: the command/payload run + a screenshot of the finding + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* confirm tooling. *Steps:* run **bash scan.sh**; confirm both Semgrep and Gitleaks sections produce output. *Deliverable:* screenshot showing both tools ran. Answer

```
qssom@Bluebell:/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling$ printf '%s | %s | ' "$(whoami)" '<6631503034>'; date '+%F %T %Z'
```

Task 1 — SAST sweep with Semgrep (25 min) · *Goal:* find code flaws. *Steps:* read the Semgrep output; locate the SQL injection in **/user** (CWE-89, string-formatted query), the OS command injection in **/ping** (CWE-78, **shell=True**), the weak **md5** password hash (CWE-327), and **debug=True** (CWE-489). *Deliverable:* one screenshot per finding with the file:line. Answer 1. SQL Injection

```
q = "SELECT * FROM users WHERE name = '%s'" % name
return str(con.execute(q).fetchall())
```

2. OS Command Injection

```
return subprocess.check_output("ping -c 1 " + host, shell=True)
```

3.

Weak MD5 Hash

```
return hashlib.md5(pw.encode()).hexdigest()
```

4. debug=True

```
app.run(debug=True) # CWE-489: debug mode in production
```

Task 2 — Secret scan with Gitleaks (15 min) · *Goal:* find leaked credentials. *Steps:* read the Gitleaks output; identify **AWS_SECRET_ACCESS_KEY** and **DB_PASSWORD** (CWE-798). *Deliverable:* screenshot + the rule that fired for each. Answer Finding: AWS_SECRET_ACCESS_KEY = "hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8" Secret: hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8 RuleID: generic-api-key Entropy: 5.080274 File: /repo/app.py Line: 11 Fingerprint: /repo/app.py:generic-api-key:11

Finding: DB_PASSWORD = "xQ7mK2pL9wR4tY6u" Secret: xQ7mK2pL9wR4tY6u RuleID: generic-api-key Entropy: 4.000000 File: /repo/app.py Line: 12 Fingerprint: /repo/app.py:generic-api-key:12

Task 3 — Bug Triage Race (30 min) · *Goal:* triage accurately. *Steps:* build a table with columns **Tool** | **File:Line** | **CWE** | **Severity** | **TP/FP** | **Fix idea**; mark at least 3 true positives and 1 likely false positive and justify each. (Score = TP – misclassified.) *Deliverable:* the completed triage table. Answer

Tool	File:Line	CWE	Severity	TP/FP	Fix idea
Semgrep	<code>app.py:19-20</code>	CWE-89	Blocking	TP	Parameterized SQL query
Semgrep	<code>app.py:26</code>	CWE-78	Blocking	TP	Remove <code>shell=True</code> ; use argument list
Semgrep	<code>app.py:30</code>	CWE-327	Blocking	TP	Use Argon2id/bcrypt
Semgrep	<code>app.py:33</code>	CWE-489	Blocking	TP	Disable debug in production
Gitleaks	<code>app.py:11</code>	CWE-798	—	TP	Environment variable/secret manager + rotate
Gitleaks	<code>app.py:12</code>	CWE-798	—	TP	Environment variable/secret manager + rotate

Triage note: Semgrep produced multiple findings for the same underlying SQL injection and command injection vulnerabilities because multiple rules detected the same code pattern. I treated these as duplicate detections rather than separate vulnerabilities. All six unique findings above are true positives based on manual inspection of `app.py`.

Task 4 — Fuzzing intro (10 min) · *Goal:* see coverage-guided fuzzing find a bug SAST won't. *Steps:* in the `labs/toolbox` container (Apple clang has no libFuzzer runtime), build `clang -g -fsanitize=address,fuzzer harness.c -o fuzz`, then **seed the corpus** and run it: `mkdir -p corpus && printf 'FUZ' > corpus/seed && ./fuzz corpus`. It crashes almost immediately with an AddressSanitizer heap-buffer-overflow at `harness.c:23` (the `data[3]` read with no `size > 3` check). Seeding matters: an unseeded `./fuzz` has to rediscover the magic bytes by chance and often finds nothing for minutes — that unpredictability is itself worth a sentence in your write-up. (The deep fuzzing+exploit lab is Week 11.) *Deliverable:* the ASan crash output (or a screenshot) + a 2-sentence note on why fuzzing finds this bug when a linter/SAST pass over the same 4-line check would not. Answer

```
qssom | 6631503034 | 2026-08-16 13:29:22 +07
qssom@Bluebell:/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling$ ./fuzz corpus
INFO: Running with entropic power schedule (0xFF, 100).
INFO: Seed: 4142394214
INFO: Loaded 1 modules (9 inline 8-bit counters): 9 [0x55df3ed66e90, 0x55df3ed66e99],
INFO: Loaded 1 PC tables (9 PCs): 9 [0x55df3ed66ea0,0x55df3ed66f30],
INFO: 1 files found in corpus
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096 bytes
INFO: seed corpus: files: 1 min: 3b max: 3b total: 3b rss: 31Mb
=====
==3572==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x71264fce0053 at pc 0x55df3ed1e707 bp 0x7fff04a39d20 sp 0x7fff04a39d18
READ of size 1 at 0x71264fce0053 thread T0
==3572==WARNING: invalid path to external symbolizer!
==3572==WARNING: Failed to use and restart external symbolizer!
#0 0x55df3ed1e706 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x151706) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#1 0x55df3ec1b85f (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x4e85f) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#2 0x55df3ec1ae89 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x4de89) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#3 0x55df3ec1cca2 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x4fca2) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#4 0x55df3ec1d1bb (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x501bb) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#5 0x55df3ec0a190 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x3d190) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#6 0x55df3ec34e96 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x67e96) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
#7 0x75065082a600 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a600) (BuildId: 240c8909736b31f963346aca80667fd00c551e32)
#8 0x75065082a717 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a717) (BuildId: 240c8909736b31f963346aca80667fd00c551e32)
#9 0x55df3ebfe994 (/mnt/d/software-security/software-security-1/labs/week02-sdlc-tooling/fuzz+0x31994) (BuildId: 6b013ad9120a21b94145b9c4eac5f83fa4cbcd5)
```

Task 5 — Scan the project target (40 min) · *Goal:* apply the tools to your term project. *Steps:* run Semgrep + Gitleaks against **NoteVault** (`../../project/starter-app`); also run an SCA scan: `docker run --rm -v "$PWD/../../project/starter-app:/src" aquasec/trivy fs /src`. *Deliverable:* a findings list (tool, file:line/CVE, CWE) — reuse it in your project vuln report. Answer Findings List

Tool	File / CVE	Finding	CWE
Semgrep	Dockerfile:12	Container runs as root / missing USER	CWE-250
Semgrep	app.py:68-69	MD5 used for password hashes	CWE-328
Semgrep	app.py:83	JWT accepts none algorithm	CWE-327
Semgrep	app.py:106-107	<code>render_template_string()</code> / possible SSTI-XSS	CWE-1336
Semgrep	app.py:117	MD5 used as password hash	CWE-916
Semgrep	app.py:128-130	SQL query constructed using user input	CWE-89
Semgrep	app.py:134	Hardcoded JWT secret	CWE-798
Semgrep	app.py:136	Insecure session cookie configuration	CWE-614
Semgrep	app.py:176-179	SQL injection in note search	CWE-89
Semgrep	app.py:181-182	User input inserted into HTML	CWE-79
Semgrep	app.py:202	Command injection via <code>subprocess(..., shell=True)</code>	CWE-78
Semgrep	app.py:204	User-controlled output returned as HTML	CWE-79
Semgrep	app.py:209	Flask debug=True / exposed host	CWE-489
Gitleaks	/src	No secrets detected	—
Trivy	requirements.txt	32 dependency vulnerabilities	—

Trivy Severity Levels Package CVEs Severity

Flask 2.0.1 CVE-2023-30861, CVE-2026-27205 HIGH, LOW Jinja2 3.0.1 CVE-2024-22195, CVE-2024-34064, CVE-2024-56201, CVE-2024-56326, CVE-2025-27516 MEDIUM PyJWT 1.7.1 CVE-2022-29217, CVE-2026-32597, CVE-2026-48526 HIGH Werkzeug 2.0.1 CVE-2023-25577, CVE-2024-34069, CVE-2023-46136, CVE-2024-49766, HIGH/MEDIUM/LOW CVE-2024-49767, CVE-2025-66221, CVE-2026-21860, CVE-2026-27199, CVE-2023-23934 requests 2.25.1 CVE-2023-32681, CVE-2024-35195, CVE-2024-47081, CVE-2026-25645 MEDIUM urllib3 1.26.4 CVE-2021-33503, CVE-2023-43804, CVE-2025-66418, CVE-2025-66471, HIGH/MEDIUM CVE-2026-21441, CVE-2026-44431, CVE-2023-45803, CVE-2024-37891, CVE-2025-50181

Gitleaks:output no leaks found

Task 6 — Build a security CI gate (25 min) · *Goal:* automate the scan (previews Week 15). *Steps:* adapt `../week15-devsecops-pipeline/security-ci.yml` into a workflow that runs Semgrep + Trivy + Gitleaks and **fails on HIGH/CRITICAL**; run it locally (**act**) or commit to your fork and read the Actions log. *Deliverable:*

the workflow file + a screenshot of a failing run. Answer

Manually triggered 1 minute ago Status: **Failure** Total duration: **47s** Artifacts: **1**

security-ci.yml
on: workflow_dispatch

- ✓ SAST (Semgrep) 26s
- ✓ SCA + image scan (Trivy) 33s
- ✗ Secret scanning (Gitleaks) 10s
- ✗ Tests (live-quiz platform) 40s

Secret scanning (Gitleaks) summary

Gitleaks detected secrets

Rule ID	Commit	Secret URL	Start Line	Author	Date	Email	File
curl-auth-header	74c95f3	View Secret	107	Nutthakorn Chalaemwongwan	2026-07-26T09:02:43Z	nutthakorn.ch@kmitl.ac.th	docs/lesson-plans/week15-devsecops-pipeline.md
curl-auth-header	74c95f3	View Secret	115	Nutthakorn Chalaemwongwan	2026-07-26T09:02:43Z	nutthakorn.ch@kmitl.ac.th	docs/lesson-plans/week15-devsecops-pipeline.md

Job summary generated at run-time

Annotations
1 error and 1 warning

- ✗ Tests (live-quiz platform)
Process completed with exit code 1.
- ⚠ Secret scanning (Gitleaks)
Leaks detected, see job summary for details

Task 7 — SAST blind spots (20 min) · Goal: see what scanners miss. Steps: find one real bug in `vulnerable-repo/app.py` (or NoteVault) that Semgrep did **not** flag, and explain why a pattern-based tool missed it.

Deliverable: the bug + a 2-sentence explanation. Answer

```

13
14 @app.route("/user")
15 def user():
16     name = request.args.get("name", "")
17     con = sqlite3.connect("app.db")
18     # CWE-89: SQL injection (string formatting into query)
19     q = "SELECT * FROM users WHERE name = '%s'" % name
20     return str(con.execute(q).fetchall())
21
22 Nutthakorn Chalaemwongwan, last month • Restructure course to 19 weeks with midterm/fin...

```

problems 29 Output Debug Console **Terminal** Ports Spell Checker 29

```

PS D:\software-security\software-security\labs\week02-sdlc-tooling> Write-Host "$(whoami) | 6631503034 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

Bug: Missing authorization / unauthenticated access to user data in `vulnerable-repo/app.py`, `/user` endpoint. The `/user` endpoint allows anyone to request user information without checking authentication or authorization. Semgrep did not flag this because missing authorization depends on application-level logic and context; a pattern-based SAST tool can detect dangerous code patterns such as SQL injection, but it generally cannot determine whether a particular user is allowed to access a particular database record. CWE: CWE-862 — Missing Authorization OWASP: A01 — Broken Access Control

Task 8 — Defend / fix it (10 min) · Goal: remediate the planted flaws in `vulnerable-repo/app.py`. Steps: rewrite `/user` to use a parameterized query (`?` placeholder); remove `shell=True` and pass an argument list in `/ping`; move both secrets to environment variables; replace `md5` with `bcrypt/argon2`; set `debug=False`.

Deliverable: a before/after diff for each fix mapped to its CWE. Answer

A	B	C	D	E
CWE	Vulnerability	Before	After	Mitigation
CWE-89	SQL Injection	String-formatted SQL query using user input	Parameterized query using ? and (name,)	Separates SQL syntax from user-controlled data
CWE-78	OS Command Injection	shell=True with concatenated user input	Argument list without shell=True	Prevents user input from being interpreted as shell syntax
CWE-798	Hardcoded Secrets	AWS secret and DB password stored directly in source code	Loaded using environment variables	Keeps credentials outside the source code
CWE-327	Weak MD5 Password Hash	hashlib.md5()	bcrypt.hashpw() with gensalt()	Uses a password-specific hashing algorithm instead of MD5
CWE-489	Debug Mode	debug=True	debug=False	Prevents Flask debug functionality from being exposed

Part 4 — Reflection

1. Map two of your findings to their CWE and to the matching OWASP 2025 category. Answer SQL Injection maps to CWE-89 and OWASP 2025 A05 Injection because user-controlled input was directly included in an SQL query. The hardcoded AWS and database credentials map to CWE-798 and OWASP 2025 A02 Security Misconfiguration because sensitive credentials were stored directly in the source code.
2. Name a real-world breach caused by a hardcoded/leaked secret or an injection flaw, and what control would have caught it pre-release. Answer A real-world example is the 2021 Codecov supply-chain incident, where attackers compromised Codecov's uploader environment and obtained credentials that could be used to access customer CI environments. Secret scanning, credential rotation, least-privilege access, and CI security controls could have reduced the risk and detected exposed credentials earlier.
3. Which single tool (SAST vs. secret scanning) gave the highest-value findings on this repo, and why? Answer SAST gave the highest-value findings on this repository because it identified several different classes of code vulnerabilities, including SQL injection, command injection, weak password hashing, and debug mode. Secret scanning was also valuable because it found the hardcoded credentials, but SAST revealed more types of exploitable application-level weaknesses in this particular repository.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (scan output + triage table + screenshots)	40
Defense (remediated <code>app.py</code> with before/after diffs)	25
Reflection (CWE/OWASP mapping + breach + tool value)	15

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` in the same image as the evidence. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (graded on your reasoning, not copied text):

1. What did you do, and **why did the vulnerability work?** Answer

```
labs > week02-sdlc-tooling > vulnerable-repo > app.py
You, 1 hour ago | 2 authors (Nuttakorn Chalaemwongwan and one other)
1
2 Deliberately INSECURE sample for Week 2 scanning practice.
3 Do NOT copy these patterns into real code. Find them with SAST + secret scanning.
4
5 import sqlite3, subprocess
6 import os
7 import bcrypt
8 from flask import Flask, request
9
10 app = Flask(__name__)
11
12 AWS_SECRET_ACCESS_KEY = os.environ.get("AWS_SECRET_ACCESS_KEY")
13 DB_PASSWORD = os.environ.get("DB_PASSWORD")
14
15 @app.route("/user")
16 def user():
17     name = request.args.get("name", "")
18     con = sqlite3.connect("app.db")
19     q = "SELECT * FROM users WHERE name = ?"
20     return str(con.execute(q, (name,)).fetchall())
21
22 @app.route("/ping")
23 def ping():
24     host = request.args.get("host", "127.0.0.1")
25     return subprocess.check_output(["ping", "-c", "1", host])
26
27 def store_password(pw):
28     return bcrypt.hashpw(pw.encode(), bcrypt.gensalt()).decode()
29
30 if __name__ == "__main__":
31     app.run(debug=False) You, 1 hour ago • Uncommitted changes
```

Problems 31 Output Debug Console Terminal Ports Spell Checker 31

```
PS D:\software-security\software-security\labs\week02-sdlc-tooling> git --no-pager diff -- vulnerable-repo/app.py
-     return subprocess.check_output("ping -c 1 " + host, shell=True)
+     return subprocess.check_output(["ping", "-c", "1", host])

def store_password(pw):
-     # CWE-327: weak hash for passwords
-     return hashlib.md5(pw.encode()).hexdigest()
+     return bcrypt.hashpw(pw.encode(), bcrypt.gensalt()).decode()

if __name__ == "__main__":
-     app.run(debug=True) # CWE-489: debug mode in production
+     app.run(debug=False)
\ No newline at end of file
• PS D:\software-security\software-security\labs\week02-sdlc-tooling> Write-Host "$(whoami) | 6631503034 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

2. Why does your fix actually stop it — and what could still break it? Answer

```

\ No newline at end of file
PS D:\software-security\software-security\labs\week02-sdlc-tooling> Write-Host "$(whoami) | 6631503034 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

```

desktop-29nrjfq\user | 6631503034 | 2026-08-23 15:30:28 +07:00
PS D:\software-security\software-security\labs\week02-sdlc-tooling> git --no-pager diff -- vulnerable-repo/app.py
diff --git a/labs/week02-sdlc-tooling/vulnerable-repo/app.py b/labs/week02-sdlc-tooling/vulnerable-repo/app.py
index 572bb89..8caf960 100644
--- a/labs/week02-sdlc-tooling/vulnerable-repo/app.py
+++ b/labs/week02-sdlc-tooling/vulnerable-repo/app.py
@@ -2,32 +2,30 @@
Deliberately INSECURE sample for Week 2 scanning practice.
Do NOT copy these patterns into real code. Find them with SAST + secret scanning.
"""

import sqlite3, hashlib, subprocess
+import sqlite3, subprocess
+import os
+import bcrypt
from flask import Flask, request

app = Flask(__name__)

-# CWE-798: hardcoded credentials / secret (Gitleaks should flag this)
-AWS_SECRET_ACCESS_KEY = "hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8"
-DB_PASSWORD = "xQ7mK2pL9wR4tY6u"
+AWS_SECRET_ACCESS_KEY = os.environ.get("AWS_SECRET_ACCESS_KEY")
+DB_PASSWORD = os.environ.get("DB_PASSWORD")

@app.route("/user")
def user():
    name = request.args.get("name", "")
    con = sqlite3.connect("app.db")
    - # CWE-89: SQL injection (string formatting into query)
    - q = "SELECT * FROM users WHERE name = '%s'" % name
    - return str(con.execute(q).fetchall())
    + q = "SELECT * FROM users WHERE name = ?"
    + return str(con.execute(q, (name,)).fetchall())

@app.route("/ping")
def ping():
    host = request.args.get("host", "127.0.0.1")
    - # CWE-78: OS command injection (shell=True with user input)
    - return subprocess.check_output("ping -c 1 " + host, shell=True)
    + return subprocess.check_output(["ping", "-c", "1", host])

def store_password(pw):
    - # CWE-327: weak hash for passwords
    - return hashlib.md5(pw.encode()).hexdigest()
    + return bcrypt.hashpw(pw.encode(), bcrypt.gensalt()).decode()

if __name__ == "__main__":
    - app.run(debug=True) # CWE-489: debug mode in production
    + app.run(debug=False)
\ No newline at end of file
PS D:\software-security\software-security\labs\week02-sdlc-tooling>

```

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer. Answer q = "SELECT * FROM users WHERE name = ?" return str(con.execute(q, (name,)).fetchall())
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s). Answer The AI fix correctly prevents the SQL injection, but it only addresses the SQL injection and does not address the missing authorization issue in the /user endpoint. An authenticated user could still potentially access another user's information if the application does not enforce authorization.
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient. Answer @app.route("/user") def user(): name = request.args.get("name", "") con = sqlite3.connect("app.db") q = "SELECT * FROM users WHERE name = ?" return str(con.execute(q, (name,)).fetchall())

The parameterized query prevents the input from changing the SQL syntax. However, authorization must also be implemented separately so that

users can only access records they are permitted to view.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon. Answer The /ping endpoint takes a hostname from the user's request and passes it to the operating system's ping command. It is exploitable because the original code concatenates the user's input into a shell command and uses shell=True, allowing special shell characters in the input to change what command the server executes.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).* Answer Fix the SQL injection vulnerability in vulnerable-repo/app.py at the /user endpoint. The current code constructs an SQLite query by formatting the user-controlled name value directly into the SQL string. Rewrite only this database query using SQLite parameterized queries with the ? placeholder and pass the value separately to con.execute(). Preserve the endpoint's existing behavior, do not use string concatenation or formatting for the query, and provide the exact before/after code plus a verification method showing that the input is treated as data rather than SQL syntax.